sesa lab
SOFTWARE ENGINEERING
SALERNO

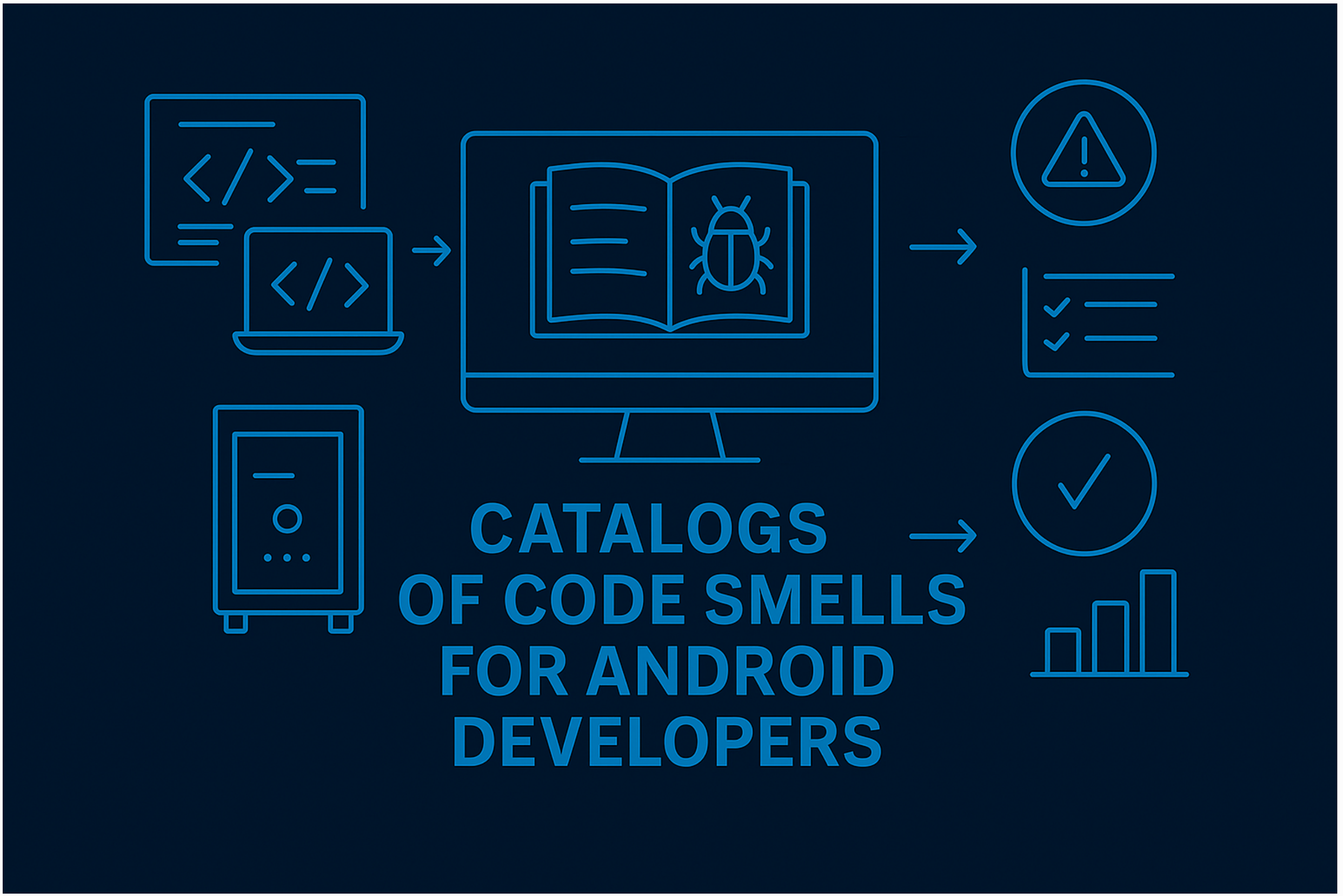
sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Energy Patterns Atelier



Slides courtesy of Dario Di Nucci | 2026 - 2027

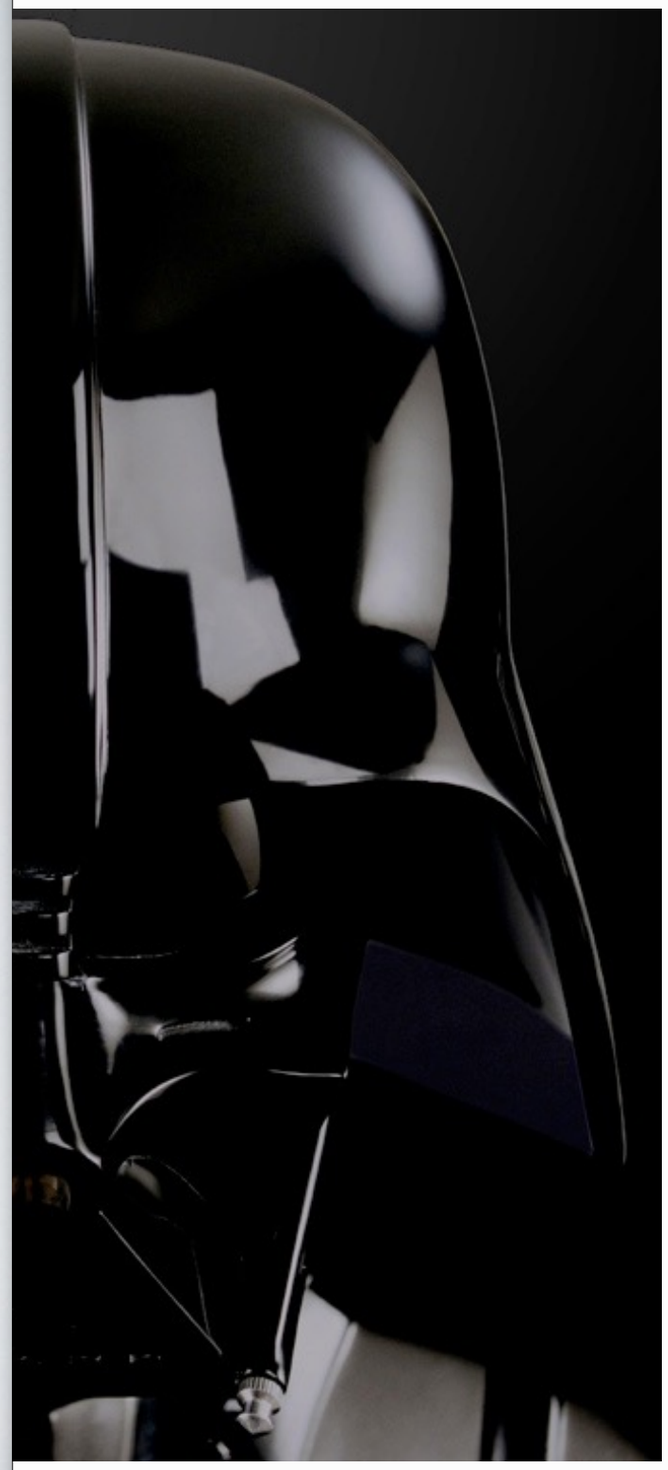
How to Improve Energy Efficiency?



How to Improve Energy Efficiency?



How to Improve Energy Efficiency?



2011 15th European C

An Empi
Antipat
Or

Marwen Abbes[1,3], Fouts
[1] Dépt. d'Informatique et de Rech
[2] Dept. of Elec. and Comp. I
[3] Ptidej Team, SOCCER
E-mails: marwen
yann-gael.

***Abstract*—Antipatterns are "poor" solutio**
design problems which are conjectured in
make object-oriented systems harder to mai
little quantitative evidence exists to support
We performed an empirical study to inve
the occurrence of antipatterns does indeed a
standability of systems by developers during
and maintenance tasks. We designed and
experiments, with 24 subjects each, to coll
performance of developers on basic tasks rel
comprehension and assessed the impact of
and of their combinations: Blob and Spag
measured the developers' performance with
task load index for their effort; (2) the
spent performing their tasks; and, (3) their
correct answers. Collected data show that th
one antipattern does not significantly decr
performance while the combination of two
pedes significantly developers. We conclude
can cope with one antipattern but that
antipatterns should be avoided possibly th
and refactorings.

***Keywords*-Antipatterns, Blob, Spaghetti**
Comprehension, Program Maintenance, Em
Engineering.

I. INTRODUCTION

Context: In theory, antipatterns are "
to recurring design problems; they stem fr
software developers' expertise and describ
falls in object-oriented programming, *e.g.*,
tipatterns [1]. Antipatterns are generally int
tems by developers not having sufficient k
or experience in solving a particular prob
misapplied some design patterns. Coplien
an antipattern as "something that looks lik
but which back-fires badly when applied
antipatterns relate to and manifest them
smells in the source code, symptoms of
and–or design problems [3].

An example of antipattern is the Bl
God Class. The Blob is a large and com
centralises the behavior of a portion of
only uses other classes as data holders, *i.*
The main characteristic of a Blob class a
a low cohesion, some method names rec

1534-5351/11 $26.00 © 2011 IEEE
DOI 10.1109/CSMR.2011.24

Authorized licensed use limited to: Eindhoven University

Empir Software Eng (2012) 17:243–275
DOI 10.1007/s10664-011-9171-y

An exploratory study of the impact of antipatterns on class change- and fault-proneness

Foutse Khomh · Massimiliano Di Penta · Yann-Gaël Guéhéneuc · Giuliano Antoniol

Published online: 6 August 2011
© Springer Science+Business Media, LLC 2011
Editor: Jim Whitehead

Abstract Antipatterns are poor design choices that are conjectured to make object-oriented systems harder to maintain. We investigate the impact of antipatterns on classes in object-oriented systems by studying the relation between the presence of antipatterns and the change- and fault-proneness of the classes. We detect 13 antipatterns in 54 releases of ArgoUML, Eclipse, Mylyn, and Rhino, and analyse (1) to what extent classes participating in antipatterns have higher odds to change or to be subject to fault-fixing than other classes, (2) to what extent these odds (if higher) are due to the sizes of the classes or to the presence of antipatterns, and (3) what kinds of changes affect classes participating in antipatterns. We show that, in almost all releases of the four systems, classes participating in antipatterns are more change- and fault-prone than others. We also show that size alone cannot explain the higher odds of classes with antipatterns to underwent a (fault-fixing) change than other

We thank Marc Eaddy for making his data on faults freely available. This work has been partly funded by the NSERC Research Chairs in Software Change and Evolution and in Software Patterns and Patterns of Software.

F. Khomh (✉)
Department of Electrical and Computer Engineering,
Queen's University, Kingston, ON, Canada
e-mail: foutse.khomh@queensu.ca

M. D. Penta
Department of Engineering, University of Sannio, Benevento, Italy
e-mail: dipenta@unisannio.it

Y.-G. Guéhéneuc · G. Antoniol
SOCCER Lab. and Ptidej Team, Département de Génie Informatique et Génie Logiciel,
École Polytechnique de Montréal, Montréal, QC, Canada

Y.-G. Guéhéneuc
e-mail: yann-gael.gueheneuc@polymtl.ca

G. Antoniol
e-mail: antoniol@ieee.org

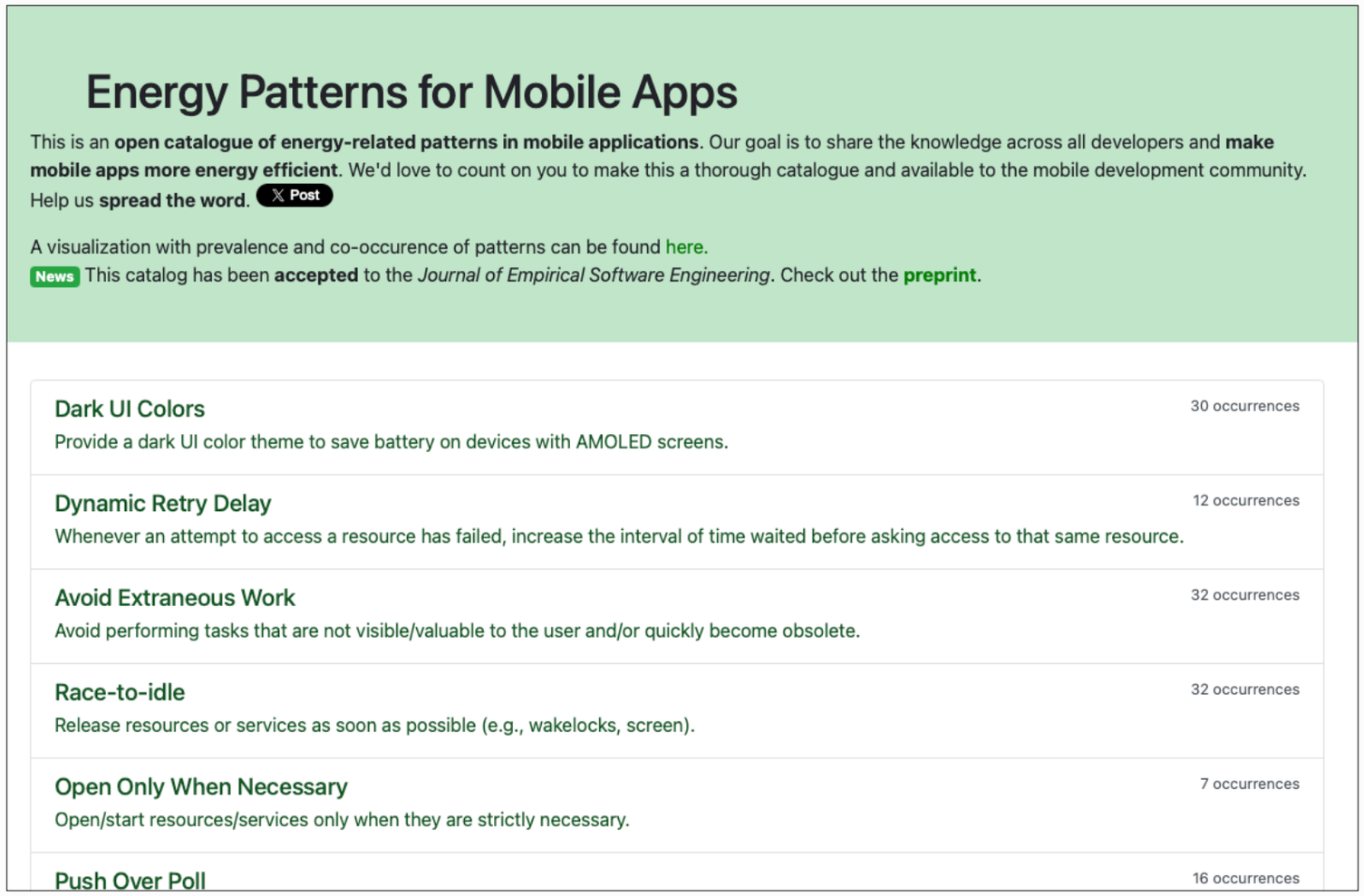
Bad Smells hinder code comprehensibility.

Bad Smells increase change and fault proneness.

The Mobile Case: Catalogs of Code Smells for Android Developers

Cruz and Abreu proposed a catalog of 22 design patterns focused on enhancing the energy efficiency of mobile apps.

The catalog is particularly relevant to Cyber-Physical Systems and the Internet of Things.
As a supplementary finding, an analysis of the differences between Android and iOS devices reveals that the Android community is more energy-conscious than the iOS community.



Cruz, Luis, and Rui Abreu. "Catalog of energy patterns for mobile applications." Empirical Software Engineering 24.4 (2019): 2209-2235.

https://tqrg.github.io/energy-patterns/

The Mobile Case: Catalogs of Code Smells for Android Developers

Reimann et al. identified a set of poor programming practices among Android developers. They relate to non-functional aspects of mobile apps, such as energy and memory efficiency.

Android Smells | Catalogue | About | Search | Submit

Table of smells and affected qualities

Smell	Accessibility	Efficiency	Energy Efficiency	Memory Efficiency	Security	Stability	Startup Time	User Conformity	User Experience
Bulk Data Transfer On Slow Network			✖					✖	
Data Transmission Without Compression			✖						
Debuggable Release					✖				
Dropped Data								✖	✖
Durable WakeLock			✖						
Early Resource Binding			✖						
Inefficient Data Structure		✖							
Inefficient SQL Query		✖							
Inefficient Data Format And Parser		✖							
Internal Getter/Setter		✖							
Interrupting From Background								✖	✖
Leaking Inner Class				✖					
Leaking Thread				✖					
Member-Ignoring Method		✖							
Nested Layout		✖					✖		✖
Network & IO Operations In Main Thread									✖
No Low Memory Resolver				✖		✖			✖
Not Descriptive UI	✖							✖	✖
Overdrawn Pixel		✖							

Reimann, Jan, Martin Brylski, and Uwe Aßmann. "A tool-supported quality smell catalogue for android developers."
Proc. of the conference Modellierung 2014 in the Workshop Modellbasierte und modellgetriebene Softwaremodernisierung–MMSM. Vol. 2014. 2014.

https://martinbrylski.github.io/android_smells/

How to Remove Code Smells? Refactoring

There are many reasons to perform refactoring, including removing code smells.

But this is just one reason, such as improving the design and quality of the code.

Refactoring aims to improve code understanding, enhance robustness, reorganize the development team, prepare for future enhancements, and decrease maintenance costs.

When?

- When you write new code from scratch to meet the expected requirements.
- When you duplicate code without regard to the source code design;
- Periodically, based on what you have developed so far.

Fowler, Martin. Refactoring: improving the design of existing code. Addison-Wesley Professional, 2018.

Are Code Smells Diffused in the Mobile Apps?

Several years ago, we analyzed the occurrences of these smells in open-source apps.

We observed that four code smells happen more often:

- Leaking Thread;
- Member Ignoring Method;
- Slow Loop;
- Data Transmission Without Compression.

The last three, along with the Internal Setter, occur more often.

Leaking Thread — A thread is created but is **not properly terminated or released** when it is no longer needed. The thread may remain alive in the background, unnecessarily consuming CPU, memory, and therefore energy.

Member Ignoring Method — A method belongs to a class but **does not use any instance fields or methods of that class**. In other words, it does not depend on the object's state. This may indicate unnecessary object interaction or poor design and, in some contexts, can introduce avoidable resource overhead.

Slow Loop — A loop performs **inefficient or unnecessarily expensive operations during each iteration**, causing it to take longer and consume more CPU resources and energy than necessary. For example, repeatedly performing the same computation inside a loop instead of computing it once beforehand.

Data Transmission Without Compression — Data is transmitted over a network **without being compressed**, even when compression could significantly reduce its size. This increases the amount of data transferred, potentially increasing network usage, transmission time, and energy consumption on both the client and server.

Palomba, Fabio, et al. "On the impact of code smells on the energy consumption of mobile applications."
Information and Software Technology 105 (2019): 43-55.

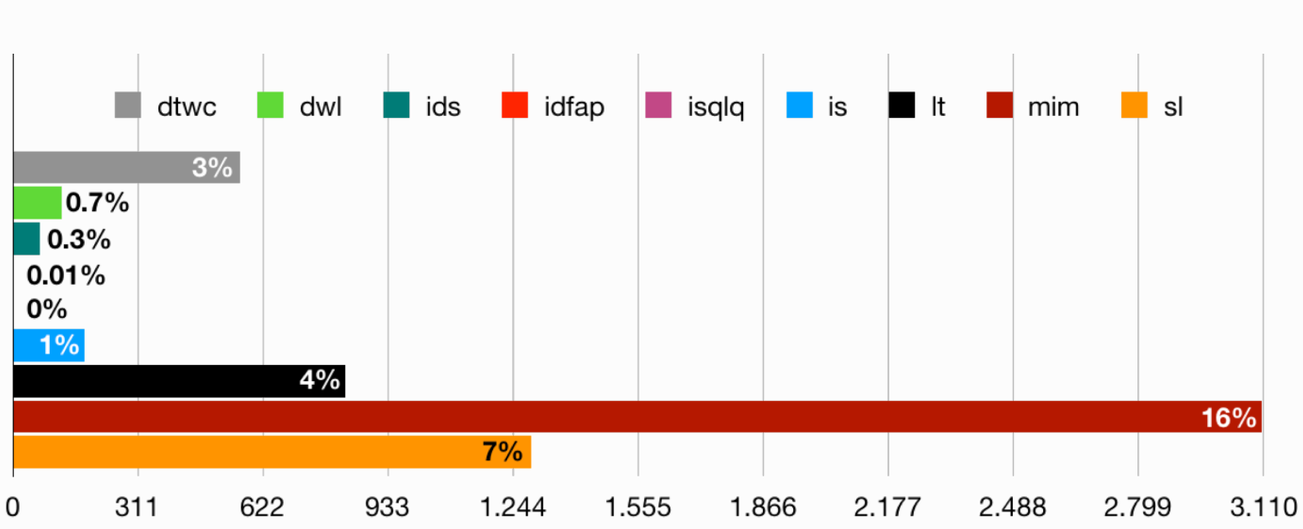
Are Code Smells Diffused in the Mobile Apps?

Several years ago, we analyzed the occurrences of these smells in open-source apps.

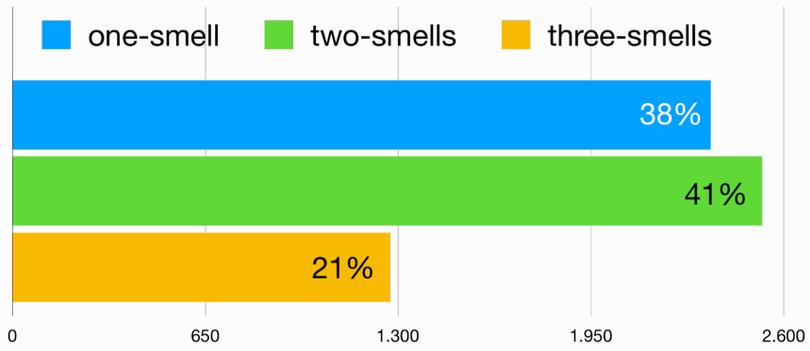
We observed that four code smells happen more often:

- Leaking Thread;
- Member Ignoring Method;
- Slow Loop;
- Data Transmission Without Compression.

The last three, along with the Internal Setter, occur more often.



Code smell diffuseness



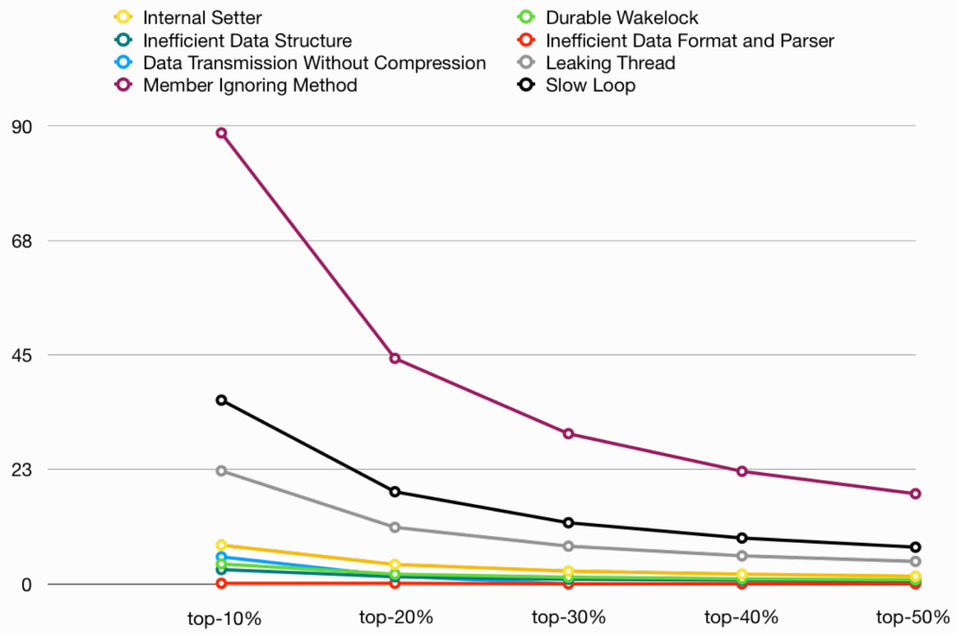
Code smell co-occurrences

Palomba, Fabio, et al. "On the impact of code smells on the energy consumption of mobile applications." Information and Software Technology 105 (2019): 43-55.

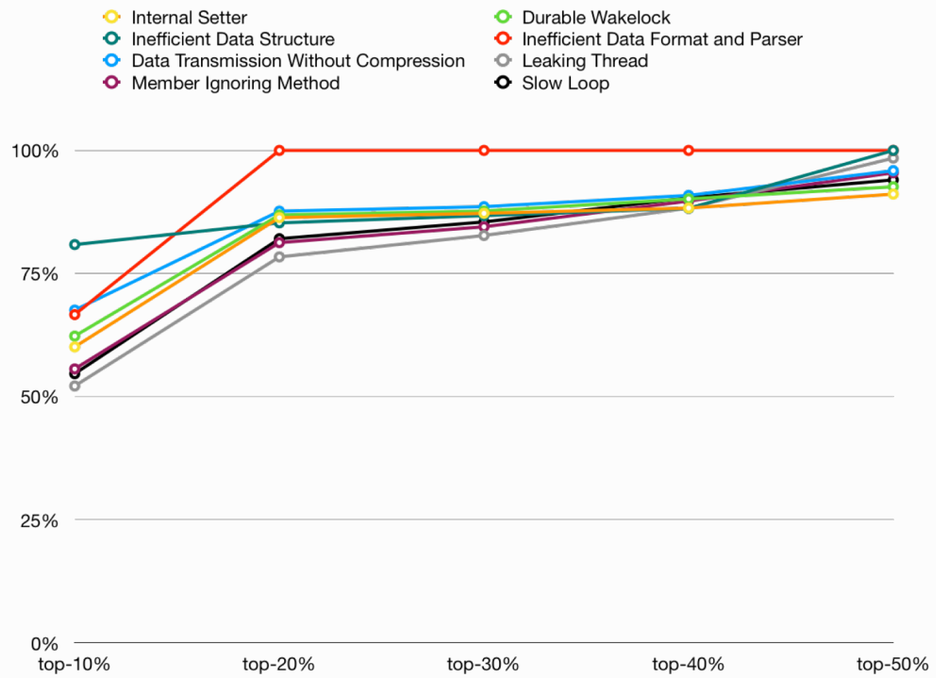
Do Methods Affected by Code Smells Have High Energy Consumption?

94% of the most greedy methods were affected by at least one code smell, indicating a strong link between code smells and energy consumption.

Greedy Methods — Methods that request or process more data/resources than they actually need to perform their task



Percentage of the most greedy methods also affected by a code smell



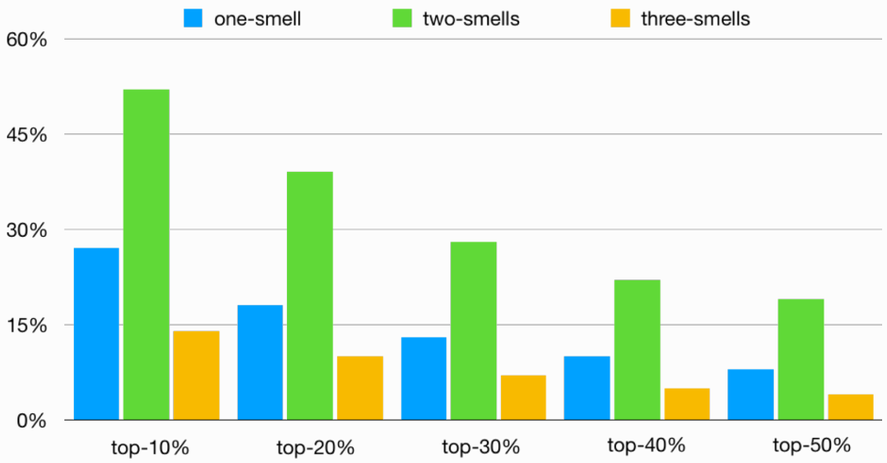
Percentage of code smells appearing in the top-10%, top-20%, top-30%, top-40%, and top-50% most greedy methods

Palomba, Fabio, et al. "On the impact of code smells on the energy consumption of mobile applications." Information and Software Technology 105 (2019): 43-55.

Does Code Smell Refactoring Impact the Energy Consumption of Mobile Apps?

Code smell refactoring is essential to enhance the energy efficiency of source code methods.

We found that the refactored versions of methods previously affected by Internal Setter, Leaking Thread, Member Ignoring Method, and Slow Loop consume up to 87 times less energy than the original methods affected by these smells.



Percentage of the most greedy methods affected by one or more smells.

Table 3: Energy consumed by methods before and after the application of refactorings. "R-" prefix stands for Refactored.

Smell Type	Min	1st Qu.	Median	Mean	3rd Qu.	Max
IS	0.076	0.076	0.082	0.083	0.092	0.092
R-IS	0.001	0.001	0.009	0.016	0.024	0.024
LT	0.01	0.02	0.03	0.077	0.013	0.77
R-LT	0.001	0.001	0.003	0.009	0.009	0.019
MIM	0.001	0.002	0.004	0.04	0.028	0.981
R-MIM	0.0001	0.002	0.018	0.02	0.019	0.038
SL	0.001	0.006	0.0119	0.056	0.026	0.929
R-SL	0.001	0.004	0.0114	0.010	0.014	0.018

Table 4: Average Battery Discharge of Methods Before and After Refactored

Smell Type	% of Battery Discharge Before	% of Battery Discharge After
IS	$2.8 \cdot 10^{-6}\%$	$3.1 \cdot 10^{-7}\%$
LT	$1.1 \cdot 10^{-6}\%$	$1.1 \cdot 10^{-7}\%$
MIM	$1.4 \cdot 10^{-6}\%$	$6.2 \cdot 10^{-7}\%$
SL	$2.1 \cdot 10^{-6}\%$	$3.9 \cdot 10^{-7}\%$

Palomba, Fabio, et al. "On the impact of code smells on the energy consumption of mobile applications." Information and Software Technology 105 (2019): 43-55.

Refactoring should be Automated!

Android code smells can remain in the codebase for years before being removed.

It only takes 34 effective commits to remove 75% of Android code smells.

Android code smells disappear faster in bigger projects with higher releasing trends.

Code smells detected and prioritized by linters tend to disappear before other code smells.

On the Survival of Android Code Smells in the Wild

Sarra Habchi
Inria / University of Lille
Lille, France
sarra.habchi@inria.fr

Romain Rouvoy
University of Lille / Inria / IUF
Lille, France
romain.rouvoy@inria.fr

Naouel Moha
Université du Québec à Montréal
Montréal, Canada
moha.naouel@uqam.ca

***Abstract*—The success of smartphones and app stores have contributed to the explosion of the number of mobile apps proposed to end-users. In this very competitive market, developers are rushed to regularly release new versions of their apps in order to retain users. Under such pressure, app developers may be tempted to adopt bad design or implementation choices, leading to the introduction of code smells. Mobile-specific code smells represent a real concern in mobile software engineering. Many studies have proposed tools to automatically detect their presence and quantify their impact on performance. However, there remains—so far—no evidence about the lifespan of these code smells in the history of mobile apps. In this paper, we present the first large-scale empirical study that investigates the survival of Android code smells. This study covers 8 types of Android code smells, 324 Android apps, $255k$ commits, and the history of $180k$ code smell instances. Our study reports that while in terms of time Android code smells can remain in the codebase for years before being removed, it only takes 34 effective commits to remove 75% of them. Also, Android code smells disappear faster in bigger projects with higher releasing trends. Finally, we observed that code smells that are detected and prioritised by linters tend to disappear before other code smells.**

***Index Terms*—Mobile apps, Android, code smells.**

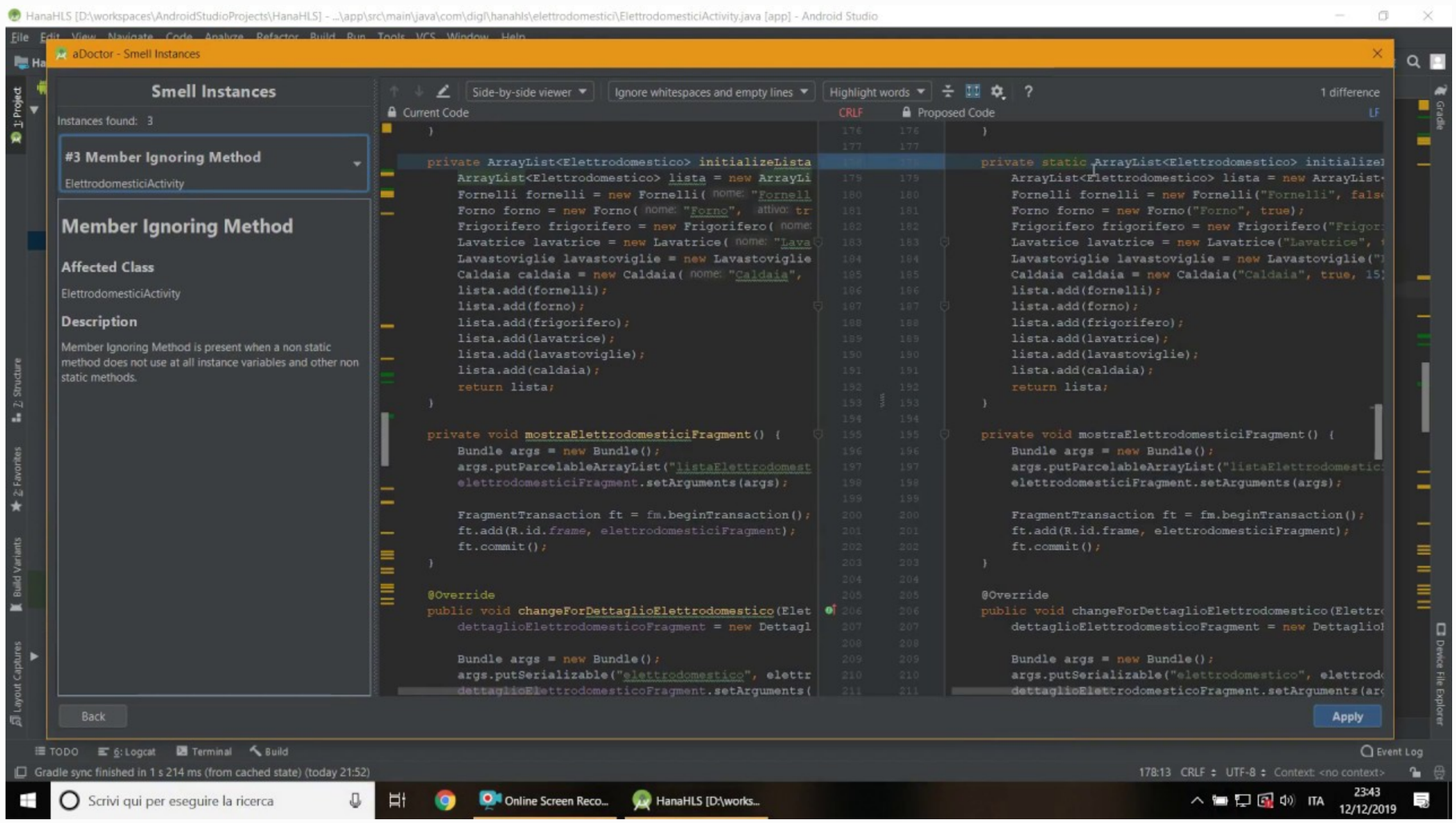
the identification of mobile-specific code smells [48]. Mobile code smells are different from OO code smells as they often refer to a misuse of the platform SDK and they are more performance-oriented. This performance aspect drew the attention of the research community as many studies investigated the impact of these code smells on performances, demonstrating that their refactoring can significantly improve app performance [9], [23], [44]. Apart from performance, other aspects of mobile-specific code smells remained unaddressed. Specifically, we still lack knowledge about the lifespan of mobile code smells in the apps history. This knowledge is particularly important for understanding the extent and importance of these code smells. Effectively, a code smell that persists in the software history for long periods can be more critical than code smells that disappear in only a few commits. Moreover, the survival analysis of these code smells allows us to understand the factors that favour or prevent their expansion in source code. Understanding such factors is crucial for proposing solutions that handle mobile code smells. For these reasons, we investigate in this paper the survival of

Habchi, Sarra, Romain Rouvoy, and Naouel Moha. "On the survival of Android code smells in the wild."
2019 IEEE/ACM 6th International Conference on Mobile Software Engineering and Systems (MOBILESoft). IEEE, 2019.

aDoctor: AnDrOid Code smell detecTOR

aDoctor is a tool to detect 15 of the Android-specific code smells identified by Reimann et al.

It is executable via terminal and a graphical user interface, and released under the MIT License.



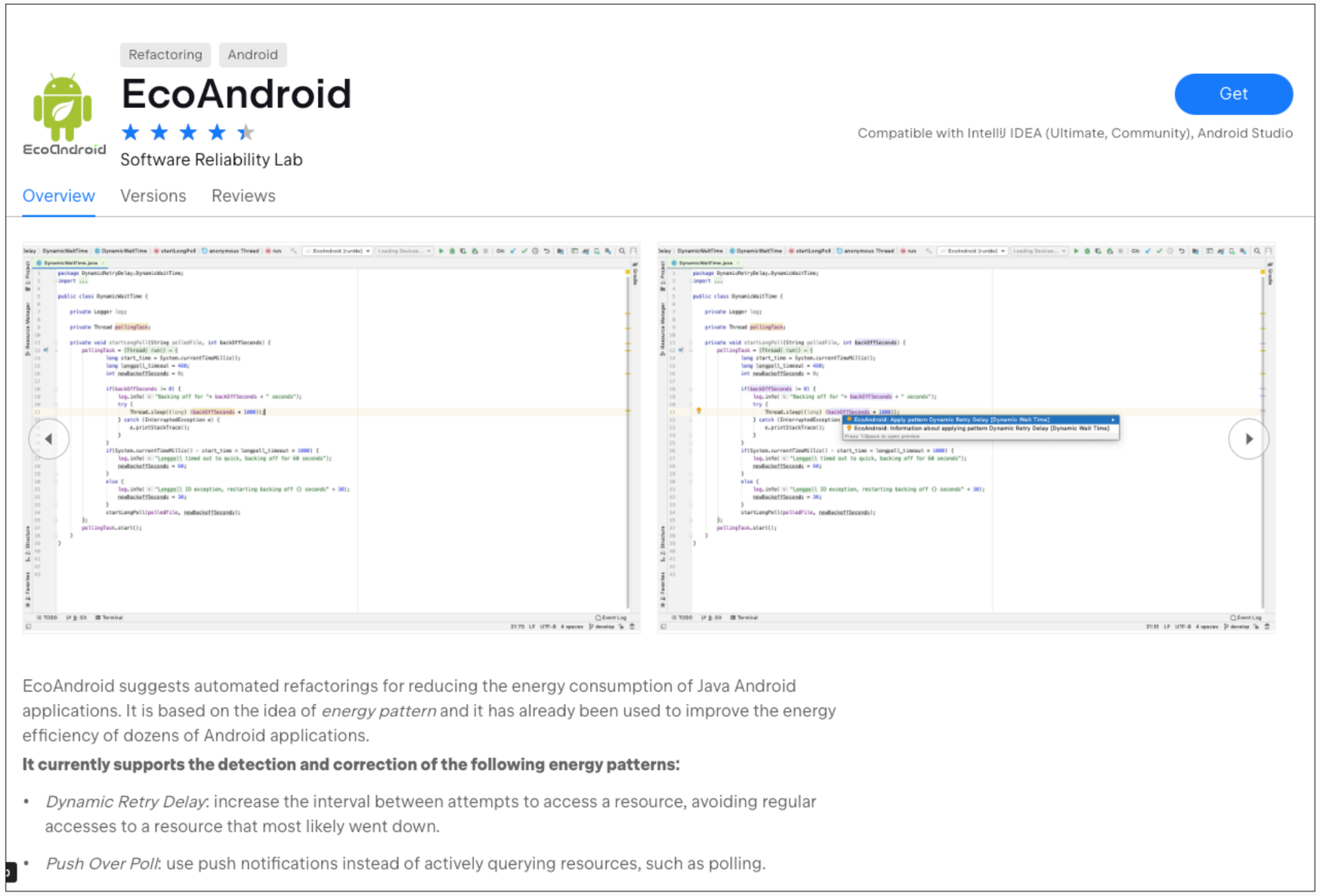
Palomba, Fabio, et al. "Lightweight detection of android-specific code smells: The adoctor project."
2017 IEEE 24th international conference on software analysis, evolution and reengineering (SANER). IEEE, 2017.

Iannone, Emanuele, et al. "Refactoring android-specific energy smells: A plugin for android studio."
Proceedings of the 28th international conference on program comprehension. 2020.

https://github.com/fpalomba/aDoctor
https://github.com/sesalab/aDoctor
https://plugins.jetbrains.com/plugin/13443-adoctor
https://www.youtube.com/watch?v=1c2EhVXiKis

EcoAndroid: An Android Studio Plugin Energy-Efficient Android Applications

EcoAndroid is an Android Studio plugin that detects ten energy anti-patterns across five categories in Android apps and refactors them.



Ribeiro, Ana, João F. Ferreira, and Alexandra Mendes. "Ecoandroid: An android studio plugin for developing energy-efficient java mobile applications." 2021 IEEE 21st international conference on software quality, reliability and security (QRS). IEEE, 2021.

https://github.com/sr-lab/EcoAndroid/
https://plugins.jetbrains.com/plugin/15637-ecoandroid

The Web Case: Catalogs of Code Smells for Web Developers

Digital activities make up 40% of our annual sustainable budget, covering greenhouse gas (GHG) emissions and the depletion of non-renewable resources.

Eco-designing a website involves analyzing its environmental impact throughout its entire lifecycle, from initial specifications to functional and graphic design, and through its final use, including development, maintenance, and ongoing operations. It is a comprehensive approach to thinking.

Eco-design also improves the efficiency of digital services.

The eco-design approach for a digital service is grounded in five key principles:

- Defining the service's functional unit, which is its main user goal.
- Assessing the environmental impact of all components required for the service implementation...
- for every lifecycle stage...
- based on various environmental indicators, such as resource depletion, water use, and emissions.
- Following a process of continuous improvement by focusing on the main enhancements first.

The Green IT Collective has compiled a set of 115 best practices for eco-designing connected digital services, including websites, APIs, and SaaS business software.

https://rweb.greenit.fr/en
https://rweb.greenit.fr/en/fiches

How to Analyze and Improve the Energy Efficiency of Web Pages?

WebsiteCarbon and EcoIndex estimate a website's carbon emissions. This free tool aims to raise awareness and inspire a more sustainable internet.

Creedengo is a collaborative project focused on minimizing the environmental impact of software at the code level. The project aims to compile a list of static code analyzers that identify potentially harmful code patterns.

GreenIT-Analysis is a browser extension that allows you to measure the environmental impacts of an entire user journey. The tool also verifies the use of best practices to reduce these impacts.

Energibridge is a cross-platform energy measurement tool that works on Linux, Windows, and macOS, as well as Intel, AMD, and Apple ARM CPU architectures.

https://www.websitecarbon.com/
https://www.ecoindex.fr/en/
https://github.com/green-code-initiative/creedengo-java
https://chromewebstore.google.com/detail/mofbfhffeklkbebfclfaiifefjflcpad?utm_source=item-share-cb
https://addons.mozilla.org/en-US/firefox/addon/greenit-analysis/
https://github.com/tdurieux/EnergiBridge

sesa lab
SOFTWARE ENGINEERING
SALERNO

sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Energy Patterns Atelier



Slides courtesy of Dario Di Nucci | 2026 - 2027